

Unit 6 and 7:

Unit 6. State Management on ASP.NET Core Application

State Management on stateless HTTP, Server-side strategies: Session State, TempData, Using HttpContext, Cache Client-side strategies: Cookies, Query Strings, Hidden Fields

Unit 7. Client-side Development in ASP.NET Core

Common client-side web technologies, JQuery, Forms and Validation, Single Page Application (SPA) Frameworks: Angular, React

By: Yuba Raj Devkota

Net Centric Computing

CSIT 6th Semester

NCCS College

Stateless Property of HTTP

HTTP is **stateless**, meaning the web server **does not remember the previous request** made by a client.

- Each HTTP request is treated as a **new and independent request**.
- The server does not automatically remember user information between requests.

Example:

A user logs in → sends another request → HTTP does not automatically remember that the user has already logged in.

Why is State Management Needed?

State management is needed to remember information about a user between multiple requests.

For example:

- *User login information*
- *Shopping cart items*
- *User preferences*
- *Form data*
- *Temporary messages*

Without state management, the server would treat every request as coming from a new user.

How is State Management Done in ASP.NET Core?

ASP.NET Core provides several **state management techniques**:

Type	Techniques
Server-side	Session State, TempData, HttpContext, Cache
Client-side	Cookies, Query Strings, Hidden Fields

State Management provides a way to store and retrieve user-related data across multiple HTTP requests, overcoming the stateless nature of HTTP.

Types of State Management

1. Client-Side State Management

- **ViewState:** Stores page-specific data
- **Cookies:** Small text files stored on client
- **Query Strings:** Data passed in URL

2. Server-Side State Management

- **Session State:** User-specific data for a session
- **Application State:** Global data for all users
- **Cache:** Temporary server storage

Server-Side State Management in ASP.NET Core

Server-side state management means storing user/application data on the server so that it can be accessed across different HTTP requests

1. Session State

Session stores data for a particular user on the **server**. It remains available across multiple requests during the user's session.

Example:

Example:

```
</> C#  
  
// Store data  
HttpContext.Session.SetString("Name", "Yuba");  
  
// Retrieve data  
string? name = HttpContext.Session.GetString("Name");
```

Use: Login information, shopping cart, user preferences, etc.

Note: Session must be configured using `AddSession()` and `UseSession()`.

? Sign is called nullable reference type in C#, which mean that name is allowed to contain null value.

1. Program.cs

```
</> C#  
  
var builder = WebApplication.CreateBuilder(args);  
  
builder.Services.AddControllersWithViews();  
  
// Enable Session  
builder.Services.AddSession();  
  
var app = builder.Build();  
  
app.UseStaticFiles();  
app.UseRouting();  
  
// Enable Session middleware  
app.UseSession();  
  
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");  
  
app.Run();
```

Program Example: Server-Side State Management using Session

This example stores a user's name in Session and displays it on another request.

2. HomeController.cs

```
</> C#  
  
using Microsoft.AspNetCore.Mvc;  
  
public class HomeController : Controller  
{  
    public IActionResult Index()  
    {  
        // Store data in Session  
        HttpContext.Session.SetString("Name", "Yuba");  
  
        return View();  
    }  
  
    public IActionResult Show()  
    {  
        // Retrieve data from Session  
        string? name = HttpContext.Session.GetString("Name");  
  
        ViewBag.Name = name;  
  
        return View();  
    }  
}
```

3. Index.cshtml

HTML

```
<h2>Session State Example</h2>  
  
<p>Name has been stored in Session.</p>  
  
<a href="/Home/Show">Show Name</a>
```

4. Show.cshtml

HTML

```
<h2>Server-Side State</h2>  
  
<p>Welcome, @ViewBag.Name</p>
```

Output

First page:

Session State Example

Name has been stored in Session.

Show Name

After clicking Show Name:

Server-Side State

Welcome, Yuba

2. TempData

TempData is used to store data temporarily, usually when transferring data from **one request/action to the next**.

Example:

```
// Store data  
TempData["Message"] = "Student saved successfully!";  
  
// Retrieve data  
string? msg = TempData["Message"]?.ToString();
```

After it is read, the TempData value is normally **removed automatically**.

Use: Success/error messages after form submission or redirection.

3. Using HttpContext

HttpContext provides information about the **current HTTP request and response**. It can also be used to access session, cookies, user information, request data, etc.

Example:

```
</> C#  
  
string? userAgent = HttpContext.Request.Headers["User-Agent"];
```

Or accessing the current user's identity:

```
</> C#  
  
string? username = HttpContext.User.Identity?.Name;
```

Use: Accessing request, response, user, session, cookies and other HTTP-related information.

- ***Session = Remember user data***
- ***TempData = Pass temporary data***
- ***HttpContext = Access current HTTP information***

4. Cache

Cache is a server-side storage mechanism used to temporarily store frequently used data so that it can be retrieved faster without repeatedly fetching or processing it.

```
public IActionResult Index()
{
    // Store data in cache
    _cache.Set("Message", "Hello Student!");

    // Retrieve data from cache
    string? msg = _cache.Get<string>("Message");

    return Content(msg);
}
```

First, register memory cache in `Program.cs` :

```
</> C#

builder.Services.AddMemoryCache();
```

And inject it into the controller:

```
</> C#

private readonly IMemoryCache _cache;

public HomeController(IMemoryCache cache)
{
    _cache = cache;
}
```

Output:

Hello Student!

Cache is useful for storing frequently accessed data such as database results, website settings, or commonly requested information, improving application performance.

Cache = Temporarily store frequently used data on the server for faster access.

What is TempData? How is it different from Session state and ViewData? When would you use TempData over other state management techniques

TempData is used to store data temporarily between two requests, especially when redirecting from one action to another.

When to use TempData?

Use **TempData** when you need to pass a small amount of data **from one request to the next**, particularly after a **redirect**.

Example: After saving a student record:

```
Save → Redirect to Index → Show "Student saved successfully!"
```

So, TempData = short-term data transfer between requests, while Session = longer-term user data and ViewData = current request data.

Then in the next action/view:

</> C#

```
TempData["Message"] = "Data saved successfully!";  
return RedirectToAction("Index");
```

</> C#

```
string msg = TempData["Message"]?.ToString();
```

Difference

Feature	TempData	Session State	ViewData
Lifetime	Usually until next request	Multiple requests	Current request only
Purpose	Transfer data during redirect	Store user-specific data	Pass data from Controller → View
Example	Success/error message	User login/name	Display a list in a view
Access	<code>TempData["x"]</code>	<code>Session["x"]</code> / <code>HttpContext.Session</code>	<code>ViewData["x"]</code>

Client-Side State Management

In client-side state management, the required information is stored or carried by the client/browser instead of being stored on the server. The main strategies are:

1. Cookies

A cookie is a small piece of data stored in the user's web browser. It can be used to remember information between requests.

Output:

Welcome, Yuba

Use: Remembering username, preferences, login-related information, etc.

Example:

</> C#

```
Response.Cookies.Append("username", "Yuba");
```

To read the cookie:

</> C#

```
string name = Request.Cookies["username"];
```

```
</> C#  
  
using Microsoft.AspNetCore.Mvc;  
  
public class HomeController : Controller  
{  
    public IActionResult SetCookie()  
    {  
        // Store data in client's browser  
        Response.Cookies.Append("Name", "Yuba");  
  
        return Content("Cookie saved successfully!");  
    }  
  
    public IActionResult GetCookie()  
    {  
        // Read data from client's browser  
        string? name = Request.Cookies["Name"];  
  
        return Content("Welcome, " + name);  
    }  
}
```

Program: Client-Side State Management using Cookies

Output

First, visit:

/Home/SetCookie

Output:

Cookie saved successfully!

Then visit:

/Home/GetCookie

Output:

Welcome, Yuba

A Cookie stores data on the client's browser. The browser sends the cookie back to the server with later requests.

Main point: In client-side state management, the data is stored on the client/browser, rather than on the server.

2. Query Strings

A **query string** passes data through the URL after the `?` symbol.

Example URL:

```
https://localhost:5001/Home/Profile?name=Yuba&age=42
```

In ASP.NET Core:

</> C#

```
public IActionResult Profile(string name, int age)
{
    return Content("Name: " + name + "\nAge: " + age);
}
```

Output:

Name: Yuba

Age: 42

Use: Passing small amounts of data between pages, such as IDs, search terms, or filters.

3. Hidden Fields

A **hidden field** is an HTML form field that is not visible to the user but sends data to the server when the form is submitted.

Example:

Example:

 HTML

```
<form method="post">
  <input type="hidden" name="StudentId" value="101" />
  <input type="submit" value="Submit" />
</form>
```

The server receives the value:

 C#

```
public IActionResult Submit(int StudentId)
{
    return Content("Student ID: " + StudentId);
}
```

Output:

Student ID: 101

Use: Maintaining values such as IDs or other information while submitting a form.

Example:

Maintain Student Name using Hidden Field

The idea is:

- *User enters a student name.*
- *The name is stored in a hidden field.*
- *When the form is submitted, the hidden field sends the name back to the server.*

1. Controller — StudentController.cs

```
</> C#  
  
using Microsoft.AspNetCore.Mvc;  
  
namespace HiddenFieldExample.Controllers  
{  
    public class StudentController : Controller  
    {  
        [HttpGet]  
        public IActionResult Index()  
        {  
            return View();  
        }  
  
        [HttpPost]  
        public IActionResult Index(string name)  
        {  
            ViewBag.Message = "Student Name: " + name;  
            return View();  
        }  
    }  
}
```

2. View — Index.cshtml

HTML

```
<h2>Student Information</h2>

<form method="post">

    <label>Enter Student Name:</label>
    <input type="text" name="name" />

    <!-- Hidden Field -->
    <input type="hidden" name="studentType" value="Regular Student" />

    <br /><br />

    <input type="submit" value="Submit" />

</form>

@if (ViewBag.Message != null)
{
    <h3>@ViewBag.Message</h3>
}
```

If you also want to **display the hidden value**, modify the controller:

```
</> C#

[HttpPost]
public IActionResult Index(string name, string studentType)
{
    ViewBag.Message = "Name: " + name +
                      "<br>Type: " + studentType;

    return View();
}
```

Output:

Student Information

Enter Student Name: [Ram]

[Submit]

Name: Ram

Type: Regular Student

Client Side Management

Strategy	Where data is kept/passed	Example
Cookies	Browser	Username
Query String	URL	<code>?id=101</code>
Hidden Fields	HTML form	Student ID

Server Side Management

Strategy	Purpose	Example
Session State	Stores user data for multiple requests	<code>Session.SetString()</code>
TempData	Stores data temporarily for the next request	<code>TempData["Message"]</code>
HttpContext	Provides access to current HTTP request/response and user information	<code>HttpContext.Request</code>